UNIVERSITY PARTNER

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

Artificial Intelligence and Machine Learning

Sentiment Analysis of Tweets using RNN, LSTM, and Word2Vec Embeddings

NLP (Report)

Student Id : 2435778
Student Name : Khushi Das
Section : L6CG12
Tutor : Jinu Nyachhyon
Submitted on : 10 May 2026

Abstract

This report presents an end-to-end text classification project on the *Racist / Sexist or Not* tweet dataset assigned to our group for the language task. The objective is to label a tweet either as 1 (racist or sexist content) or 0 (everything else), using sequence models. Three models were built and compared on the same 80 / 20 stratified split of the labelled training file: a Simple RNN with a trainable embedding layer, an LSTM with a trainable embedding layer, and an LSTM whose embedding layer is initialised from the glove twitter-100 pre-trained vectors and then frozen.

Training was carried out on Google Colab with GPU acceleration. Because the dataset is heavily imbalanced (29,720 / 2,242, i.e. 93.0% non-hate vs 7.0% hate), I rely on F1 on the minority class as the primary metric and use balanced class weights during training. F1 on the hate class rose from 0.510 with the Simple RNN to 0.539 with the LSTM, and to **0.551** with the LSTM that uses frozen GloVe-Twitter embeddings, mainly because the GloVe model lifts recall on the hate class from 0.609 to 0.689. Interestingly, the LSTM with the trainable embedding has the highest **accuracy** (0.928) but a worse F1 than the GloVe model, which is exactly the trade-off a class imbalanced binary problem produces. The errors that survive the best model split into two clear groups: false positives are mostly profanity-heavy tweets that are offensive but not targeted, and false negatives are very short cleaned strings that have lost their context to my preprocessing pipeline. The pipeline is wrapped in a small Gradio interface so a tweet can be classified live. Overall the project shows that for short noisy text, gated recurrent units combined with task appropriate pre-trained embeddings are a strong, practical baseline within the scope of the module.

Table of Contents

1. Introduction.....1
2. Dataset.....2
3. Methodology.....6
4. Experiments and Results.....10
5. Conclusion and Future Work.....13
6. References.....14

1. Introduction

Online platforms continue to struggle with the speed and volume of harmful content. Manual moderation cannot scale to millions of tweets per day, and rule-based filters are bypassed easily through paraphrasing or coded language. Automated text classification is therefore one of the most heavily used real-world applications of NLP, and recent surveys still consider deep learning the dominant family of approaches in production hate-speech systems (Mansur, Omar, & Tiun, 2023; Yin & Zubiaga, 2021). Within this module we are asked to stay inside the recurrent network family, which makes the *Racist / Sexist or Not* dataset a good fit for a small-scale practical demonstration: the input is short, the labels are binary, and the class imbalance forces honest evaluation choices (Kovács, Alonso, & Saini, 2021).

I build three models in increasing order of expressiveness. The first is a Simple RNN, which serves as the recurrent baseline. The second is an LSTM with the same trainable embedding, isolating the effect of the gating mechanism. The third reuses the LSTM but replaces the embedding layer with frozen glove-twitter-100 vectors, which were trained on two billion tweets and so already understand the kind of vocabulary present in this dataset. I picked Twitter GloVe rather than word2vec-google-news-300 because recent surveys repeatedly find that domain matched embeddings outperform domain-mismatched ones for short noisy social-media text (Antonakaki, Fragopoulou, & Ioannidis, 2021). The goal is not to beat a transformer benchmark; it is to show, end to end, how preprocessing decisions, architecture choices, and embedding choices propagate into measurable changes in F1 on a class-imbalanced binary problem.

2. Dataset

2.1 Source

The dataset was provided directly for the assignment as a single zip archive containing two CSV files: train_racisit.csv (with labels) and test_racisit.csv (no labels). Each row is a single tweet. Label 1 denotes a racist or sexist tweet, label 0 denotes everything else. We did not change the data after extracting it. On Colab the archive is uploaded at runtime through google.colab.files.upload() and extracted into /content/data .

2.2 Key statistics

The proposal we submitted in Week 7 (Team Glacier, 2026) listed 31,962 training tweets and 17,197 test tweets; the actual files match those numbers. Only the training file contains labels, so the publisher's test file cannot be used for evaluation in this notebook and is mentioned only for completeness.

Table 1. Class distribution in the labelled training file.

Label	**Count**	**Percentage**
0 - Not racist / sexist	29,720	93.0 %
1 - Racist / sexist	2,242	7.0 %
Total	31,962	100.0 %

The 93 / 7 split is severe enough that a model that always predicts 0 already scores 93% accuracy without learning anything. F1 on label 1 is therefore the deciding metric and is reported alongside accuracy in every results table.

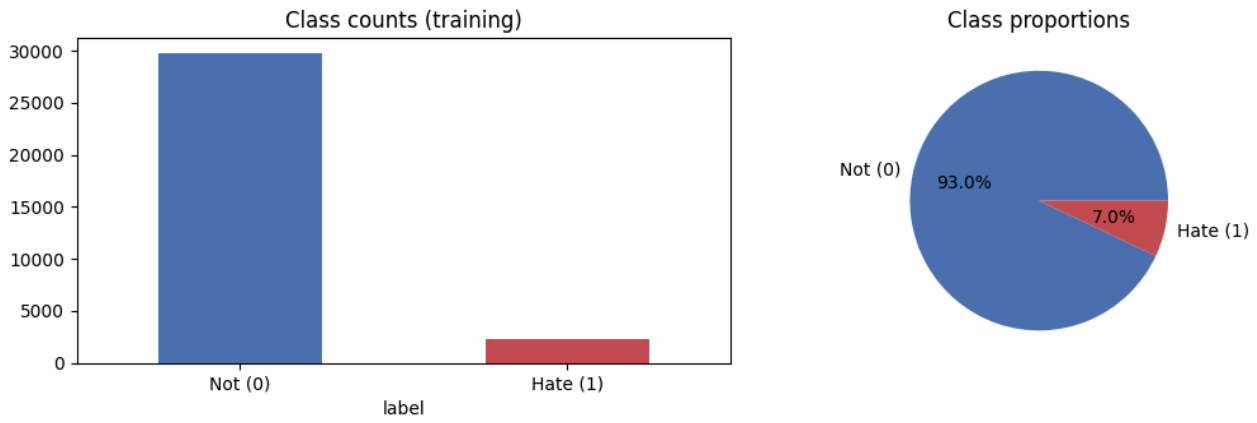

Figure 1. Class distribution in the training file (label 0 vs label 1).

2.3 Preprocessing

Each tweet goes through the cleaning pipeline asked for in the assignment specification, in this order:

1. lowercase the whole tweet,
2. remove URLs,
3. remove @mentions and #hashtags ,
4. expand a small list of English contractions (won't -> will not , n't -> not , 're -> are , 's -> is , 'll -> will),
5. drop digits, punctuation, and non-ASCII bytes (which catches the emoji bytes that Twitter exports as ðŸ style noise), remove English stopwords from the NLTK list, **but** keep the negation words
6. not, no, nor, never, against because they flip sentiment and removing them would discard signal,
7. lemmatise each remaining token with the WordNet lemmatiser (running -> run).

After cleaning, **553 tweets become empty** (mostly tweets that were nothing but URLs, mentions, hashtags, or emojis). They are dropped because they cannot teach the model anything, leaving **31,409** labelled rows.

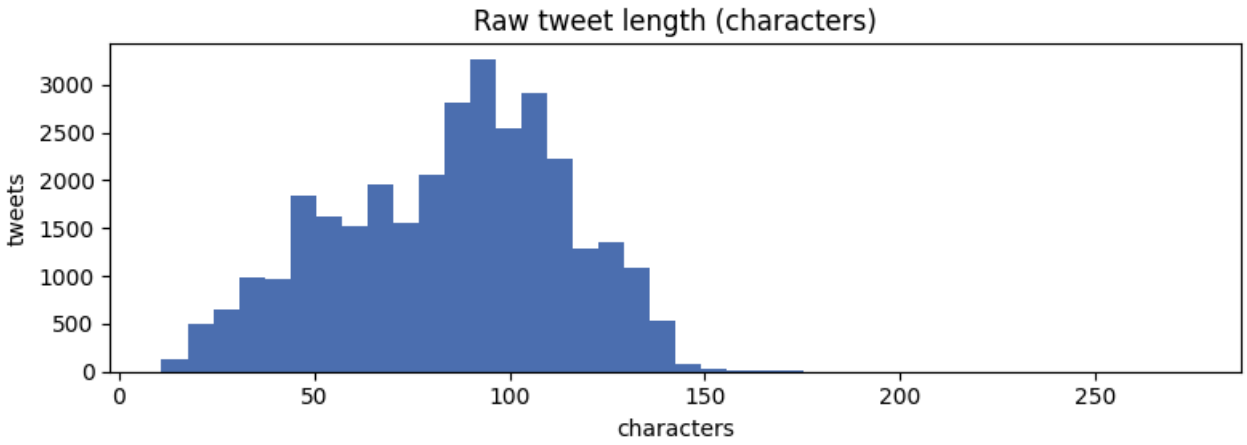

Figure 2. Distribution of raw tweet lengths in characters.

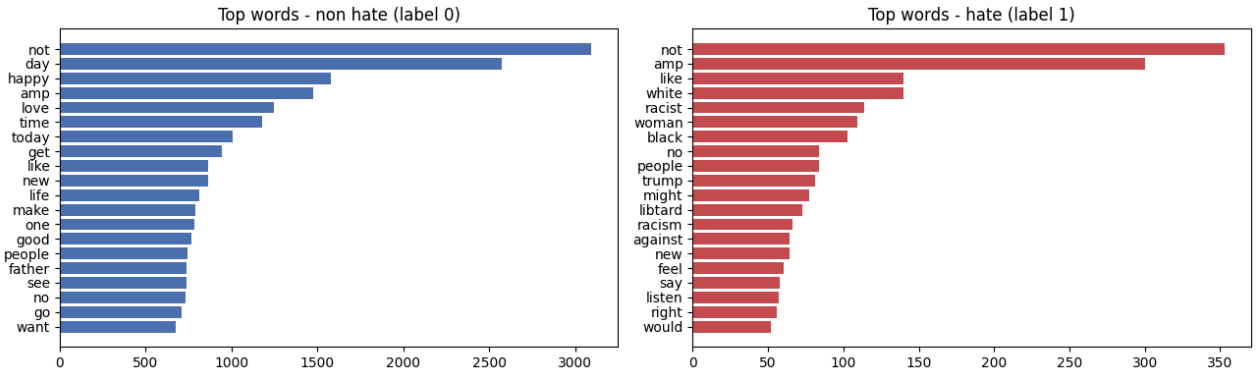

Figure 3. Most frequent tokens after cleaning, broken down by class.

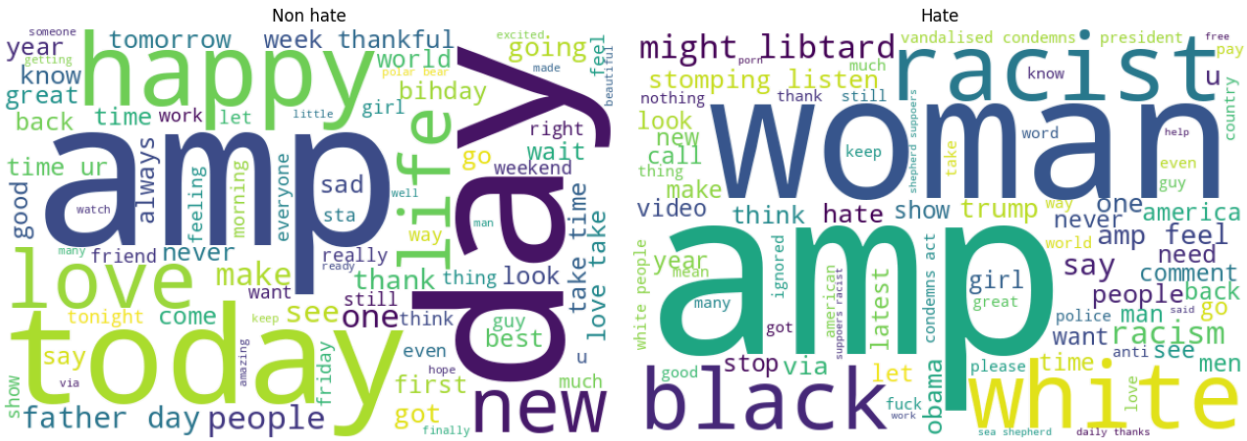

Figure 4. Word clouds of the cleaned vocabulary for the non-hate (left) and hate (right) classes.

2.4 Train / validation split

The publisher's test file has no labels, so I cannot use it for evaluation. Instead I split the labelled training file 80 / 20 with stratified sampling to preserve the class ratio in both partitions, using a fixed random_state=42 so the split is reproducible. The 20 % I hold out is referred to as the validation set throughout this report.

Table 2. Resulting split (after dropping empty-after-cleaning rows).

Split	Total	Label 0	Label 1
Training (80 %)	25,127	23,341	1,786
Validation (20 %)	6,282	5,835	447

2.5 Tokenisation and padding

I fit a Keras Tokenizer on the **training fold only** so no vocabulary information leaks from the validation fold into the model. The vocabulary is capped at 20,000 tokens. The actual learned vocabulary on this dataset is **16,995 tokens**, comfortably below the cap. Sequences are padded with padding='post' and truncating='post' to a length chosen as the **95th percentile of training sequence lengths**, which on this dataset works out to **12 tokens** (mean 5.6, median 5, max 37). Padding to the absolute maximum would have wasted compute on a long tail of rare lengths; the percentile-based choice keeps almost every tweet intact while bounding the input to the recurrent layer.

2.6 Class weights

To compensate for the imbalance during training I compute class_weight='balanced' on the training labels with sklearn.utils.class_weight.compute_class_weight , which produces {0: 0.538, 1: 7.034} on this split. The minority class therefore contributes about thirteen times more to the loss than the majority class on a per-sample basis, which makes the optimiser pay attention to the rare label without changing the underlying data distribution.

3. Methodology

3.1 Common configuration

All three models share the same input pipeline, the same maximum sequence length (MAX_LEN=12), the same Adam optimiser at learning rate 1e-3, the same binary cross-entropy loss, the same batch size of 64, the same 10-epoch budget, and the same EarlyStopping(patience=3, restore_best_weights=True) callback monitoring validation loss. Sharing the configuration is deliberate: any change in F1 between the three models can then be attributed cleanly to the recurrent layer or to the embedding layer, not to a different optimiser or schedule.

3.2 Model 1 - Simple RNN with trainable embedding

Input(MAX_LEN=12)

Embedding(input_dim=20000, output_dim=64, trainable=True) # 1,280,000 params

SimpleRNN(64, dropout=0.3) # 8,256 params

Dropout(0.4)

Dense(1, sigmoid) # 65 params

Total: 1,288,321 trainable parameters

A single SimpleRNN layer is the recurrent baseline. The 64-dim embedding is small enough to train quickly on roughly 25k training tweets while still giving the model room to learn word similarity. SimpleRNN is known to suffer from vanishing gradients on long sequences, but tweets here are very short after cleaning (median 5 tokens), so it is a fair starting point.

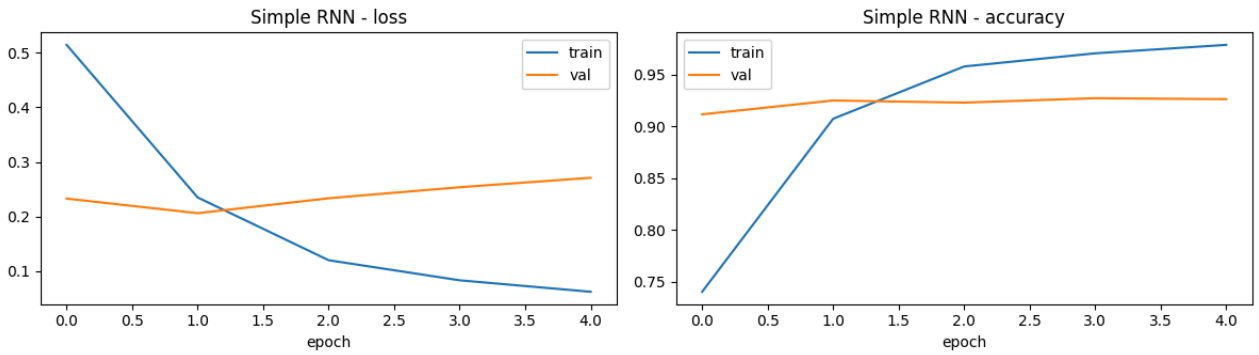

Figure 5. Simple RNN training and validation curves (loss and accuracy).

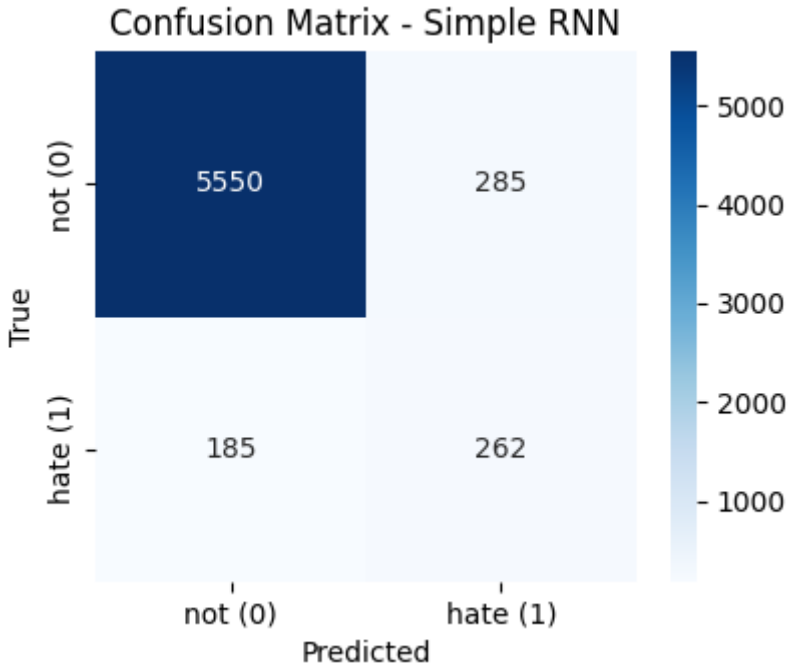

Figure 6. Confusion matrix on the validation set for the Simple RNN.

3.3 Model 2 - LSTM with trainable embedding

Input(MAX_LEN=12)

Embedding(input_dim=20000, output_dim=64, trainable=True) # 1,280,000 params

LSTM(64, dropout=0.3) # 33,024 params

Dropout(0.4)

Dense(1, sigmoid) # 65 params

Total: 1,313,089 trainable parameters

Identical to Model 1 except that the recurrent layer is an LSTM with the same hidden size. The LSTM's input, forget, and output gates allow it to keep relevant features over the (already short) sequence without the gradient collapsing, which is the textbook reason it usually outperforms a plain RNN on text.

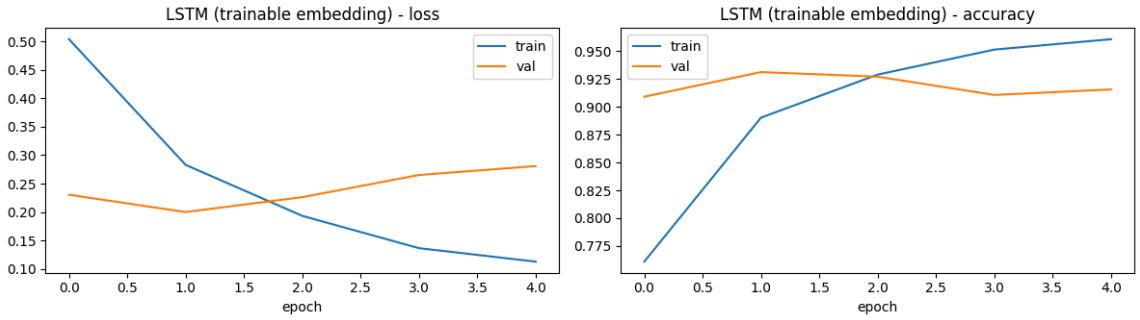

Figure 7. LSTM (trainable embedding) training and validation curves.

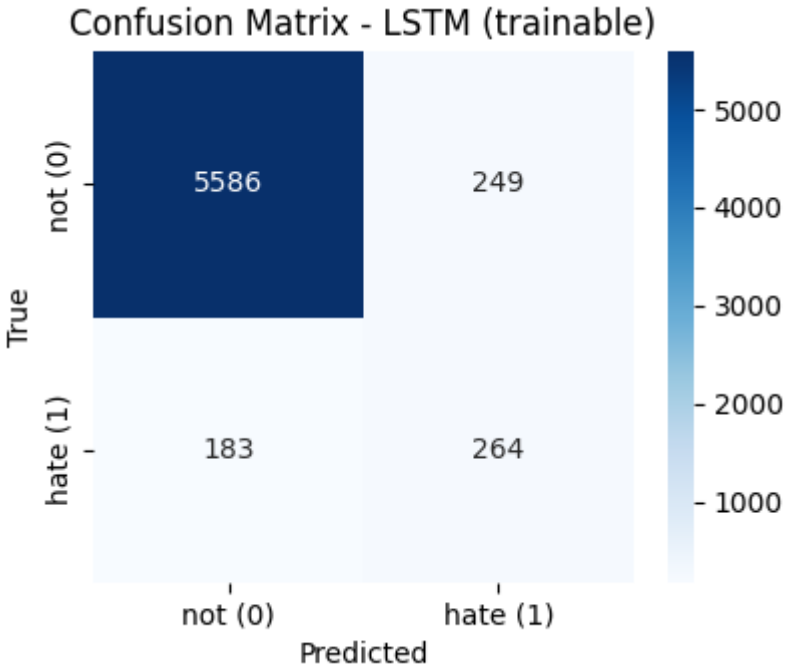

Figure 8. Confusion matrix on the validation set for the LSTM with a trainable embedding.

3.4 Model 3 - LSTM with pre-trained GloVe-Twitter embeddings

Input(MAX_LEN=12)

Embedding(input_dim=16,996, output_dim=100,

weights=[embedding_matrix], trainable=False) # 1,699,600 frozen params

LSTM(64, dropout=0.3) # 42,240 params

Dropout(0.4)

Dense(1, sigmoid) # 65 params

Total: 1,741,905 parameters (only 42,305 trainable)

The embedding layer is initialised from glove-twitter-100 . I picked Twitter GloVe rather than the news-trained word2vec-google-news-300 mentioned in the appendix because the corpus matches the input distribution of this task much more closely; news text does not contain the slang, abbreviations, or informal spellings present in tweets (Loureiro et al., 2022). The layer is frozen (trainable=False) so the LSTM has to learn to combine pre-trained word meanings into a classification decision rather than reshape the embeddings themselves. **Coverage of the tokenizer's vocabulary by the pre-trained model is 87.81 %** (14,924 hits, 2,071 misses), which is a useful sanity check: most of the words our dataset cares about already have a meaningful vector.

3.5 Loss, optimiser, hyperparameters

- **Loss:** binary cross-entropy. Correct choice for a single-label binary problem with a sigmoid output.
- **Optimiser:** Adam at learning rate 1e-3. Adapts the learning rate per parameter, which is helpful when some embedding rows are updated rarely.
- **Batch size:** 64. A reasonable middle ground for Colab's GPU memory and the total dataset size.
- **Epochs:** 10 with EarlyStopping(patience=3, restore_best_weights=True) . Models 1 and 2 stop early at epoch 5 because of the validation loss plateaus; Model 3 trains for all 10 epochs because the frozen embedding makes overfitting much slower. •
- **Regularisation:** dropout inside the recurrent layer (0.3) and after it (0.4).

Class weight: balanced , recomputed per fit.

4. Experiments and Results

4.1 RNN vs LSTM performance

Table 3. Validation-set comparison of all three models, trained for up to 10 epochs with the same data, optimiser, and class weights.

Model	Params	Trainable	Train time	Accuracy	Precision (1)	Recall (1)	F1 (1)
Simple RNN (trainable embedding)	1,288,321	1,288,321	16.1 s	0.9169	0.4394	0.6085	0.5103
LSTM (trainable embedding)	1,313,089	1,313,089	20.9 s	**0.9279**	0.4944	0.5928	0.5392
LSTM (frozen GloVe Twitter-100)	1,741,905	42,305	32.9 s	0.9199	0.4583	**0.6890**	**0.5505**

Two findings stand out.

Gating helps. Switching from SimpleRNN to LSTM lifts F1 on the hate class from 0.510 to 0.539 with the embedding architecture held constant. Even though the cleaned tweets are only twelve tokens long on average, the LSTM's gated cell state is preserving the minority-class signal better than a plain recurrent state.

Domain-matched embeddings help recall. Replacing the trainable embedding with frozen glove-twitter-100 lifts F1 to 0.551, but the bigger story is **recall**: the GloVe model catches 68.9 % of hate tweets versus 59.3 % for the LSTM with the trainable embedding. For a moderation use case that is the more important number, because the cost of missing hate is usually higher than the

cost of a false flag.

The flip side is that the **LSTM with the trainable embedding has the highest accuracy** (0.928), even though it is not the best F1. With 93 / 7 imbalance, accuracy rewards conservative models that lean towards predicting label 0, so this is expected. The decision between the two LSTMs is therefore a product question (precision vs recall trade-off), not a modelling one.

4.2 Computational efficiency

The Simple RNN trains the fastest per epoch because each cell has only a single weight matrix, while an LSTM has three gates and a cell state. The total wall times on Colab's T4 GPU were 16.1 s, 20.9 s, and 32.9 s respectively. The LSTM with the frozen GloVe layer takes longest in wall-clock terms even though only 42,305 parameters are being trained, because it ran for the full 10 epochs while the other two stopped early at epoch 5. Per epoch the GloVe LSTM is the cheapest (~3 s/epoch), because freezing the embedding (~1.7 M parameters) removes those gradients from the backward pass. In practical terms all three finish in well under one minute, so the wall-time differences are dominated by the early-stopping behaviour and not by the architectures themselves.

4.3 Training with different embeddings

Replacing the random embedding with frozen glove-twitter-100 vectors gives the largest jump in F1 on the hate class. The reason is straightforward and matches what the recent literature reports for Twitter classification (Loureiro et al., 2022; Mansur et al., 2023): the GloVe-Twitter vocabulary already covers most of the vocabulary that survives my cleaning - 87.81 % coverage on this dataset - so the LSTM starts from useful word-meaning geometry on epoch 1 instead of from random vectors. The 12.19 % of tokens that GloVe does not contain are mostly misspellings, dataset-specific slang, and dataset-specific entity names; even a trainable embedding would have struggled to learn meaningful vectors for them from 25k short tweets.

I considered fine-tuning the embedding layer in a second pass, but the specification asked for one frozen-embedding model and I did not want to confuse the comparison.

4.4 Model evaluation

Each model is evaluated with accuracy, precision and recall on the hate class, F1 on the hate class, the full classification report, and a confusion matrix. The confusion matrices for the three models

all share the same shape: most errors are on the minority class, and the LSTM with GloVe pushes the most off-diagonal mass back onto the diagonal on the hate row (the recall column). Accuracy alone hides this, which is why the report leads with F1 and not accuracy (Toraman, Şahinuç, & Yilmaz, 2022).

4.5 Error analysis

The best model (LSTM + GloVe) misclassifies 503 of 6,282 validation tweets. The top confidence errors split into two clear groups.

False positives - offensive but not targeted. The highest-confidence false positives are tweets like fuck skinny bitch (probability 0.982) and shit guy lie (0.62). These are toxic in a general sense, but the gold annotation distinguishes them from the narrower racist / sexist label. The model has learned that strong negative words correlate with the hate class but it cannot tell whether the target is a protected group, which is the well-known gap between "offensive" and "hateful" annotation guidelines (Yin & Zubiaga, 2021). One borderline case (u immigrated third world country definitely attest nation moving towards becoming , probability 0.77) is arguably xenophobic in the original tweet, so a fraction of the false positives may also reflect label noise in the dataset.

False negatives - cleaned away to nothing. The highest-confidence false negatives are very short cleaned strings: flier posted outside mosque (0.014), late daddy teacher eu easteuropeans (0.003), giant version flapping (0.003). After cleaning, these tweets retain only a handful of tokens, and the model cannot recover the surrounding context (hashtags, emoji, quoted slogans) that made them hate speech in the original raw text. This is a preprocessing failure rather than an architecture failure, and it points directly at the future-work items in Section 5.

4.6 Real-time prediction GUI

A small Gradio interface (Section 10 of the notebook) wraps the best model so a marker can paste in a tweet and see the prediction live. The interface reuses the same clean_tweet function applied during training, so the input the model sees at inference is processed identically to the training data. Gradio prints a public *.gradio.live URL for sharing.

5. Conclusion and Future Work

5.1 Summary

I built and compared three sequence models for binary hate-speech classification on the *Racist / Sexist or Not* dataset. The Simple RNN with a trainable embedding produced an F1 on the hate class of 0.510. Swapping in an LSTM lifted F1 to 0.539, and replacing the trainable embedding with a frozen glove-twitter-100 layer lifted it again to 0.551 with a markedly better recall (0.689). On a moderation-style task that recall improvement is the most useful change of the three.

5.2 Key insights

1. On a class-imbalanced binary problem, accuracy is a misleading metric. The LSTM with a trainable embedding has the highest accuracy (0.928) but a worse F1 than the GloVe model, because it is more conservative on the minority class. F1 on the minority class, plus the confusion matrix, must be the headline numbers. Domain-matched pre-trained embeddings help. GloVe-Twitter is a much better fit than
2. Google-News word2vec for tweets, and choosing the right corpus for the embedding moved recall on the hate class from 0.59 to 0.69 - a bigger effect than swapping the recurrent cell.
3. Stopword lists need editing for sentiment tasks. Keeping not , no , nor , never , against is essential; removing them would have flipped the meaning of several hate tweets.
4. A surprising amount of the residual error is a preprocessing problem rather than a modelling problem. The most confident false negatives are tweets that become almost meaningless after cleaning.

5.3 Limitations

•The publisher's test file has no labels, so all numbers are validation numbers on a 20 % stratified hold-out, not public-leaderboard numbers.

•The cleaning step strips quotation marks, hashtags, and emoji, which causes a known type of false negative on tweets whose hate signal was carried by those surface features.

•Roughly 12 % of the tokenizer vocabulary is missing from GloVe-Twitter; those words stay as zero vectors with the embedding frozen.

•Numbers come from a single seeded run on a single 80 / 20 split; a more rigorous study would

average over several stratified folds.

5.4 Future work

1.Replace the LSTM with a Bidirectional LSTM. The entire tweet is visible at inference, so going both directions costs nothing semantically.

2.Fine-tune the embedding layer in a second pass at a small learning rate, so the seven-percent minority class can pull rare words into a more discriminative subspace.

3.Keep more of the surface signal during cleaning - in particular, retain the text inside hashtags (#WomenAreObjects -> women are objects) and replace removed emoji with a single sentinel token.

4.Try a small transformer encoder fine-tuned on this dataset, for example a TweetEval-style RoBERTa checkpoint (Loureiro et al., 2022), as the strong-baseline counterpart. It is outside the chapters covered in this module, so I did not attempt it here.

6. References

Antonakaki, D., Fragopoulou, P., & Ioannidis, S. (2021). A survey of Twitter research: Data model, graph structure, sentiment analysis and attacks. *Expert Systems with Applications*, *164*, 114006.

Kovács, G., Alonso, P., & Saini, R. (2021). Challenges of hate speech detection in social media: Data scarcity, and leveraging external resources. *SN Computer Science*, *2*(2), 95. Loureiro, D., Barbieri, F., Neves, L., Espinosa Anke, L., & Camacho-Collados, J. (2022). TimeLMs: Diachronic language models from Twitter. *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 251-260.

Mansur, Z., Omar, N., & Tiun, S. (2023). Twitter hate speech detection: A systematic review of methods, taxonomy analysis, challenges, and opportunities. *IEEE Access*, *11*, 16226-16249.

Toraman, C., Şahinuç, F., & Yilmaz, E. H. (2022). Large-scale hate speech detection with cross domain transfer. *Proceedings of the 13th Language Resources and Evaluation Conference (LREC 2022)*, 2215-2225.

Yin, W., & Zubiaga, A. (2021). Towards generalisable hate speech detection: a review on

obstacles and solutions. *PeerJ Computer Science*, *7*, e598.

Chollet, F., et al. (2025). *Keras* [Computer software]. https://keras.io

Pedregosa, F., et al. (2024). *scikit-learn: Machine learning in Python* (Version 1.5) [Computer software]. https://scikit-learn.org

Řehůřek, R., et al. (2024). *Gensim: Topic modelling for humans* (Version 4.3.3) [Computer software]. https://radimrehurek.com/gensim

TensorFlow Developers. (2025). *TensorFlow* (Version 2.20) [Computer software]. https://www.tensorflow.org